

UNIVERSIDADE DO ESTADO DO PARÁ
CENTRO DE CIÊNCIAS NATURAIS E DE TECNOLOGIA – CCNT

João Vittor Carvalho Pereira
Jorge Eduardo Silva Gonçalves
Juan Pablo Lopes Pinto
Luís Miguel Frasso Silva Puty
Márcio Deivid Ferreira Pavão
Pedro Fabiano Belfort de Araújo Gondim

Projeto de Linguagens Formais: Linguagem de programação Quantix

João Vittor Carvalho Pereira
Jorge Eduardo Silva Gonçalves
Juan Pablo Lopes Pinto
Luís Miguel Frasão Silva Puty
Márcio Deivid Ferreira Pavão
Pedro Fabiano Belfort de Araújo Gondim

Projeto de Linguagens Formais: Linguagem de programação Quantix

Projeto avaliativo desenvolvido na disciplina
de Linguagens Formais, ministrada pelo
professor Anderson Costa.

SUMÁRIO

1	A LINGUAGEM QUANTIX.....	4
1.1	Alfabeto.....	6
1.1.1	Palavras reservadas.....	8
1.1.1.1	Gramáticas das estruturas do projeto.....	10
2	AUTÔMATOS.....	13
2.1	Estrutura principal do programa.....	13
2.1.1	Estruturas de decisão.....	14
2.1.1.1	Estruturas de repetição.....	15
2.1.1.1.1	Declaração de variáveis.....	16
2.1.1.1.1.1	Cálculos aritméticos.....	18
2.1.1.1.1.2	Funções.....	19
3	MÁQUINAS DE TURING.....	20
3.1	Comparação de palavras.....	20
3.1.1	Concatenação de palavras.....	21
3.1.1.1	Encontrar as letras de uma palavra em um texto.....	22

1 A LINGUAGEM QUANTIX

A presente linguagem de programação foi desenvolvida com o objetivo principal de democratizar o ensino de lógica e algoritmos, tornando-se uma ferramenta simples, intuitiva e totalmente voltada para iniciantes. Sua concepção parte da premissa de que a barreira do idioma e a complexidade sintática de linguagens tradicionais, como C, Java e Python, frequentemente desmotivam estudantes nos primeiros contatos com a programação. Para mitigar esse problema, adotou-se um conjunto de palavras-chave em português, aproximando a escrita de código da linguagem natural do aluno e reduzindo a carga cognitiva inicial.

Estrutura Geral e Organização de Blocos

Diferente de linguagens que utilizam chaves `{ }` ou indentação significativa, esta linguagem adota uma estrutura de blocos bem definida e delimitada por palavras reservadas. Todo programa inicia obrigatoriamente com a diretiva `projeto`, que nomeia a aplicação. O corpo principal do código é demarcado pelas palavras `inicio` e `fimprojeto`, garantindo que o estudante visualize claramente onde começa e termina a execução. Essa abordagem foi inspirada na clareza estrutural do Pascal e na objetividade do VisuAlg, mas com sintaxe modernizada.

Sistema de Tipos e Declaração de Variáveis

A linguagem possui tipagem explícita e estática, exigindo que todas as variáveis sejam declaradas antes do uso. Foram definidos quatro tipos primitivos para cobrir as necessidades básicas do ensino: `Int`: Representa números inteiros, como 10, -5 ou 0. Equivalente ao `int` em C/Java. `real`: representa números com casas decimais, como 3.14 ou -0.5. Equivalente ao `float` ou `double`. `txt`: Armazena cadeias de caracteres, ou seja, textos. Equivalente a `string`. `lgc`: Tipo lógico que aceita apenas os valores verdadeiro ou falso. Equivalente ao `boolean`. As regras para nomeação de identificadores proíbem o uso de palavras reservadas, acentos e caracteres especiais, incentivando boas práticas desde o início.

Operadores e Construção de Expressões

Para permitir a resolução de problemas matemáticos e lógicos, a linguagem suporta três categorias de operadores: Matemáticos: +, -, *, /, mod para resto de divisão. A precedência segue as regras convencionais da matemática. Relacionais: =, <>, >, <, >=, <= para comparações. O operador de igualdade = foi escolhido por ser mais intuitivo que == para iniciantes. Lógicos: e, ou, nao, permitindo a combinação de condições complexas de forma legível.

Comandos de Entrada e Saída

A interação com o usuário é feita através de dois comandos específicos: memo: Derivado de "memorizar", é o comando de entrada. Ele lê um valor digitado pelo usuário e armazena em uma variável. Ex: memo (idade). Substitui o scanf do C e o input do Python. mostre: Exibe informações na tela. Pode concatenar textos e variáveis. Ex: mostre ("Sua idade é ",idade). Substitui o printf e print. A escolha dos nomes memo e mostre reforça a metáfora de "conversar com o computador".

Estruturas de Controle de Fluxo

Para implementar lógica algorítmica, a linguagem oferece estruturas de controle completas: Condicional: A estrutura se <condicao> entao ... senao ... fimse permite a execução seletiva de blocos. A obrigatoriedade do fimse evita ambiguidades do "dangling else". Repetição: São oferecidas duas estruturas. O enquanto <condicao> faca ... fimenquanto executa enquanto a condição for verdadeira, ideal para laços com número indefinido de repetições.

Modularização e Reutilização de Código

Visando introduzir conceitos de engenharia de software desde cedo, a linguagem permite a definição de funções com a palavra-chave funcao. Cada função pode receber parâmetros tipados, possuir um bloco inicio...fimfuncao próprio e retornar valores com o comando retorne. Isso promove a divisão de problemas complexos em subproblemas menores, organização lógica e reutilização de código.

Conclusão e Inspirações

Em síntese, a linguagem busca unir a robustez estrutural de C, a orientação a blocos de Java e a simplicidade sintática de Python, mas com o diferencial de ser 100% em português.

1.1 Alfabeto

O alfabeto da linguagem, denotado por Σ , é composto por letras maiúsculas e minúsculas do alfabeto latino, dígitos numéricos de 0 a 9 e um conjunto definido de caracteres especiais. Esses elementos constituem os símbolos terminais fundamentais utilizados na análise léxica e formam a base para a construção de identificadores, palavras reservadas, literais, operadores e demais comandos. A combinação desses símbolos permite a elaboração de expressões aritméticas, relacionais e lógicas, garantindo flexibilidade e expressividade na escrita dos programas. $\Sigma = \{ a...z, A...Z, 0...9, / \% \wedge = < > ! \& | () \{ \} [] ; , : . " ' \ + - \}$ A definição formal do alfabeto assegura que o compilador ou interpretador reconheça corretamente todos os tokens válidos da linguagem, evitando ambiguidades durante a fase de tokenização e facilitando a detecção de erros sintáticos.

$$\Sigma = \{ a...z, A...Z, 0...9, / \% \wedge = < > ! \& | () \{ \} [] ; , : . " ' \ + - \}$$

1.1.1 Palavras reservadas

As palavras reservadas são termos fundamentais da linguagem que possuem um significado especial para o compilador e, por essa razão, não podem ser utilizadas como nomes de variáveis ou funções. Na Quantix, essas palavras são escritas em português para tornar a programação mais intuitiva para iniciantes.

As palavras reservadas são organizadas nas seguintes categorias funcionais:

1. Palavras Estruturais (Blocos de Código)

Estas palavras definem a organização global do programa e a delimitação de seus componentes principais:

- projeto: Marca o início da definição do programa.
- inicio: Delimita o começo do bloco de execução principal ou de subprogramas.
- fimprojeto: Finaliza o escopo global do programa ou de uma função.

2. Tipos de Dados e Valores Literais

Define os formatos de dados aceitos pela linguagem e seus estados lógicos:

- int: Para números inteiros.
- real: Para números reais.
- txt: Para cadeias de caracteres (strings).
- lgc: Para tipos lógicos (booleanos).
- vrdd: Valor literal que representa "Verdadeiro".
- flso: Valor literal que representa "Falso".

3. Controle de Fluxo (Decisão e Repetição)

Comandos que gerenciam a ordem e as condições de execução das instruções:

- se, entao, senao,: Estruturas de decisão condicional.
- enquanto, faca: Laço de repetição que executa enquanto uma condição for satisfeita..

- para: Laço de repetição controlado por uma variável de incremento ou decrescimento.

- escolha, caso, outrocaso, fimescolha: Estrutura de seleção múltipla para diferentes fluxos baseados em um valor.

4. Entrada/Saída e Comandos de Sistema

Comandos para interação com o usuário e controle do terminal:

- mostre: Imprime dados na tela do terminal.
- mostre_In: Variação que imprime dados e realiza um salto de linha.
- memo: Recebe dados digitados pelo teclado e armazena em uma variável.

5. Definição de Subprogramas (Funções)

- funcao: Inicia a declaração de um sub-bloco reutilizável.
- retorna: Especifica o valor de saída que a função enviará ao final de sua execução.

6. Operadores Especiais

Embora existam símbolos para estas operações, as palavras também são reservadas:

- %: Operador aritmético para calcular o resto da divisão.
- e, ou, nao, xou: Operadores lógicos de conjunção, disjunção, negação e disjunção exclusiva.

1.1.1.1 Gramáticas das estruturas do projeto

Estrutura Global e Blocos de Comandos

A recursão em <comandos> permite que o bloco inicio ... fimprojeto contenha qualquer número de instruções.

- <programa> → projeto id ; <inicio_bloco> fimprojeto
- <inicio_bloco> → inicio <comandos>
- <comandos> → <comando> <comandos> | ε (Onde ε permite que o bloco seja vazio ou finalize a sequência)
- <comando> → <declaracao> | <atribuicao> | <saida> | <entrada> | <condicional> | <estrutura_enquanto> | <para> | <estrutura_escolha> | <def_funcao>

Declarações e Atribuições

A regra <declaracoes> permite declarar múltiplas variáveis em sequência (ex: int a; real b;) antes de chegar ao inicio.

- <declaracoes> → <tipo> id <inicializacao> <declaracoes> | inicio <comandos>
- <inicializacao> → = <termo> ; | ;
- <atribuicao> → id = <expr_linear> ;
- <tipo> → int | real | txt | lgc

Cálculos Aritméticos (Expressão Linear) e Expressões

A recursão à direita em <expr_linear> permite sequências infinitas de operações (ex: a + b - c * d) de forma "achatada".

- <expr_linear> → <expr_aritmetica> | <termo_logico>
- <expr_aritmetica> → <expr_aritmetica> + <termo_mat> | <expr_aritmetica> - <termo_mat> | <termo_mat>
- <termo_mat> → <termo_mat> * <fator> | <termo_mat> / <fator> | <termo_mat> % <fator> | <termo_mat> mod <fator> | <fator>

- $\langle \text{fator} \rangle \rightarrow (\langle \text{expr_aritmetica} \rangle) \mid \text{id} \mid \text{numero}$

Estruturas de Decisão e Repetição

Para manter a regularidade, o corpo destas estruturas aponta para $\langle \text{comando} \rangle$, que pode ser um comando simples ou uma nova sequência linear.

- $\langle \text{condicional} \rangle \rightarrow \text{se } \langle \text{condicao} \rangle \text{ entao } \langle \text{comandos} \rangle \langle \text{senao_opcional} \rangle \text{ fimse}$
- $\langle \text{senao_opcional} \rangle \rightarrow \text{senao } \langle \text{comandos} \rangle \mid \epsilon$
- $\langle \text{estrutura_enquanto} \rangle \rightarrow \text{enquanto } \langle \text{condicao} \rangle \text{ faca } \langle \text{comandos} \rangle \text{ fimenquanto}$
- $\langle \text{para} \rangle \rightarrow \text{para } (\text{id} = \langle \text{expr_linear} \rangle ; \langle \text{condicao} \rangle ; \text{id} = \langle \text{expr_linear} \rangle) \langle \text{comando} \rangle \text{ fimpara}$
- $\langle \text{condicao} \rangle \rightarrow \langle \text{termo} \rangle \langle \text{op_rel} \rangle \langle \text{termo} \rangle$
- $\langle \text{op_rel} \rangle \rightarrow = \mid < \mid < \mid > \mid < = \mid > =$
- $\langle \text{estrutura_escolha} \rangle \rightarrow \text{escolha id } \langle \text{lista_casos} \rangle \langle \text{outro_caso_opcional} \rangle \text{ fimescolha}$
- $\langle \text{lista_casos} \rangle \rightarrow \text{caso } \langle \text{termo} \rangle \langle \text{comandos} \rangle \langle \text{lista_casos} \rangle \mid \text{caso } \langle \text{termo} \rangle \langle \text{comandos} \rangle$
- $\langle \text{outro_caso_opcional} \rangle \rightarrow \text{outrocaso } \langle \text{comandos} \rangle \mid \epsilon$

Entrada/Saída e Sistema

- $\langle \text{saída} \rangle \rightarrow \text{mostre } (\langle \text{expr_linear} \rangle) ; \mid \text{mostre_ln } (\langle \text{expr_linear} \rangle) \backslash \text{n};$
- $\langle \text{entrada} \rangle \rightarrow \text{memo } (\text{id}) ;$

Definição de Funções e Parâmetros

A regra `<prox_param>` garante que você possa listar múltiplos parâmetros separados por vírgula (ex: `int x, real y, txt z`).

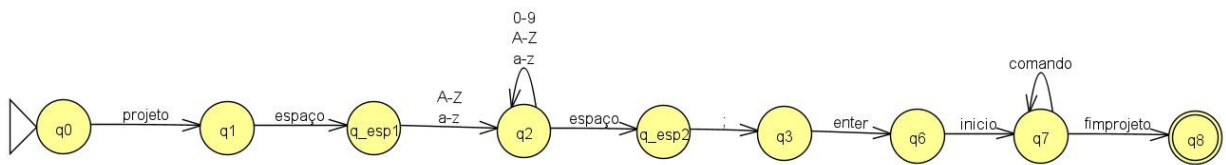
- `<def_funcao>` $\rightarrow$ `funcao id ( <parametros> ) : <tipo> <comandos> retorna <expr_linear> ;`
- `<parametros>` $\rightarrow$ `<tipo> id <prox_param> | \epsilon`
- `<prox_param>` $\rightarrow$ `, <tipo> id <prox_param> | \epsilon`
- `<comandos>` $\rightarrow$ `<comando> <comandos> | \epsilon`

2 AUTÔMATOS

A análise sintática e léxica da Quantix é fundamentada na teoria dos Autômatos Finitos Determinísticos (AFD). Como a linguagem foi projetada sob uma Gramática Livre de Contexto (Tipo 2) , cada estrutura de código é reconhecida por um sistema de estados finitos que processa tokens de forma linear.

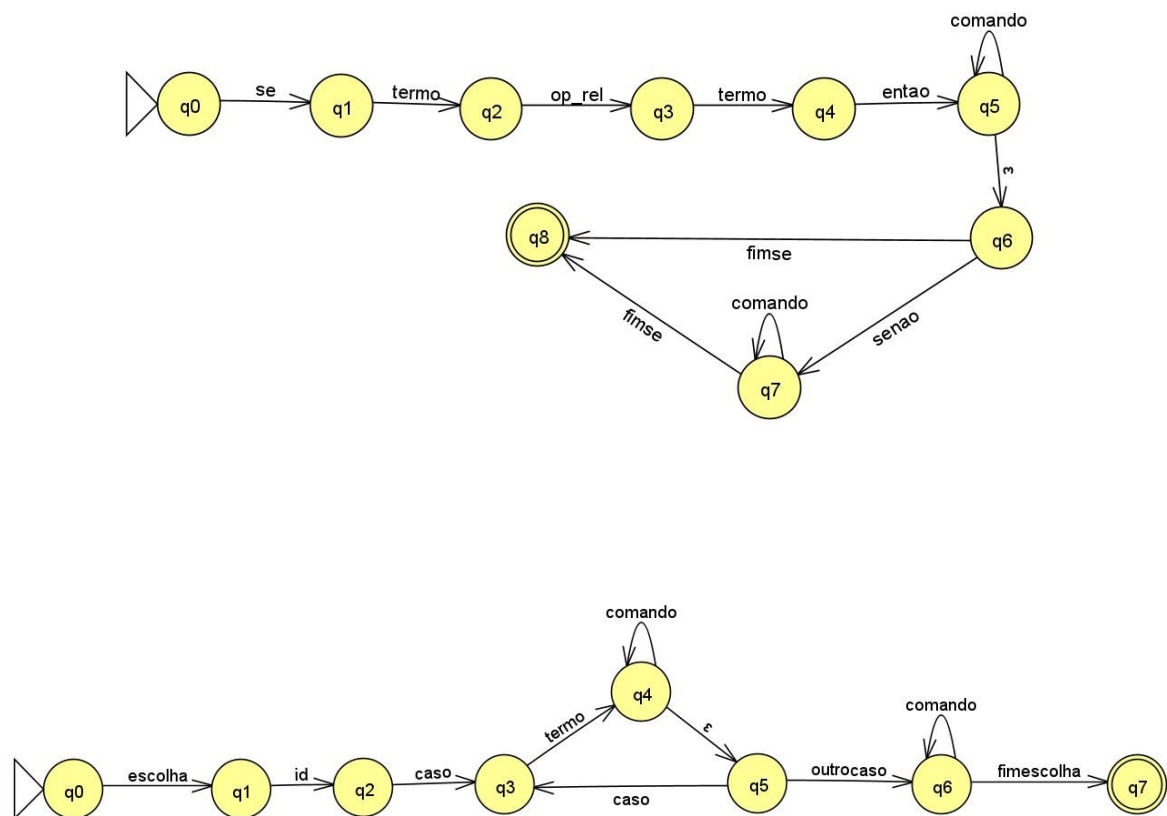
2.1 Estrutura principal do programa

Este diagrama garante a ordem obrigatória dos blocos globais, iniciando obrigatoriamente com a palavra reservada projeto e finalizando com fimprojeto.



2.1.1 Estruturas de decisão

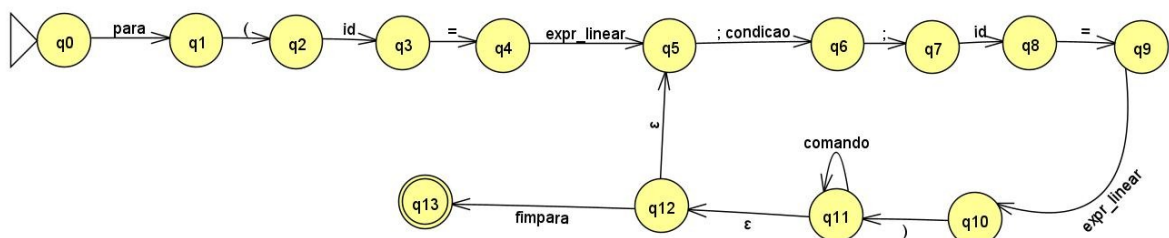
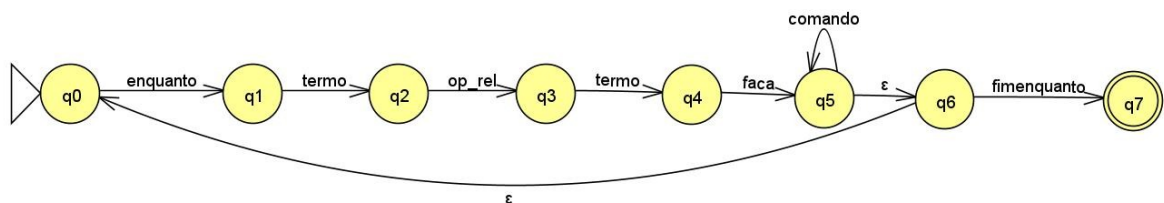
As estruturas de decisão na Quantix são fundamentais para o controle de fluxo, permitindo que o programa execute blocos específicos de código baseados na avaliação de condições lógicas. O comando principal é o *se*, que verifica uma condição; se esta for verdadeira (*vrdd*), as instruções após a palavra-chave *entao* são processadas. Caso a condição seja falsa (*flso*), a linguagem oferece a cláusula opcional *senao* para definir um caminho alternativo. A estrutura *escolha...caso* é uma estrutura de controle de fluxo condicional utilizada para simplificar múltiplos testes em uma única variável.



2.1.1 Estruturas de repetição

As estruturas de repetição (laços ou *loops*) na Quantix permitem que um conjunto de instruções seja executado múltiplas vezes, eliminando a necessidade de repetir código manualmente. A linguagem oferece duas variações para diferentes necessidades de controle:

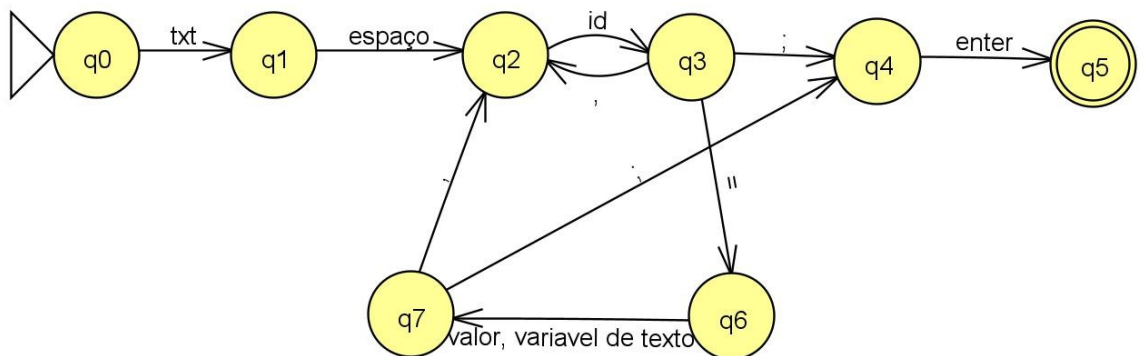
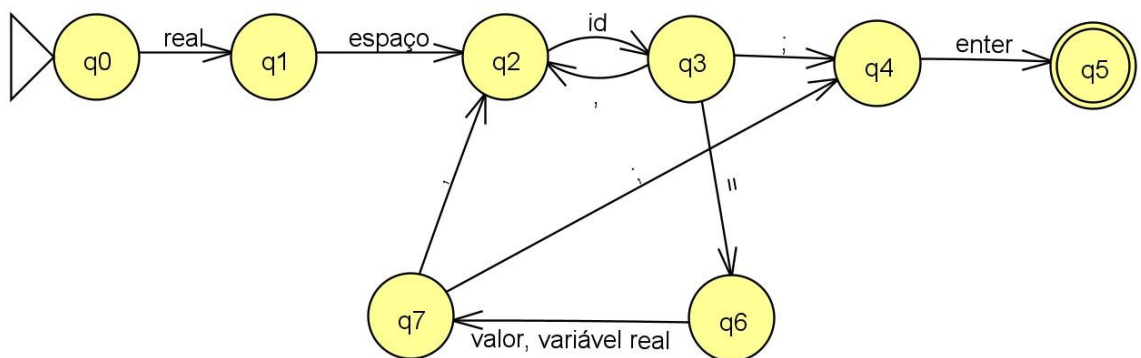
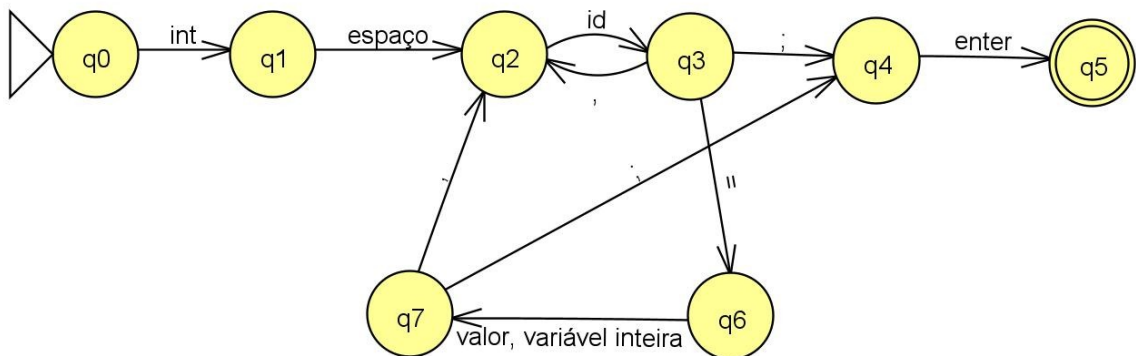
- enquanto: Verifica uma condição antes de cada execução; o bloco só é processado enquanto a condição for verdadeira (vrdd).
- para: Ideal para repetições finitas, pois consolida a inicialização, a condição e a atualização da variável de controle (incremento ou decrescimento) em uma única linha.

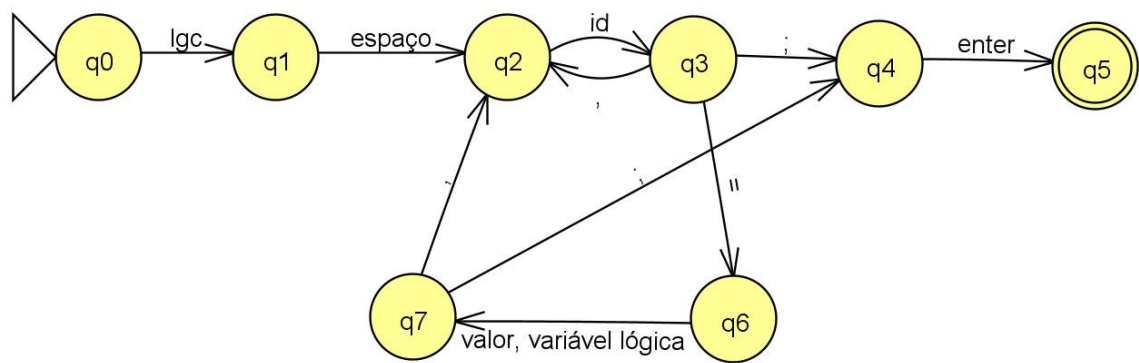


2.1.1.1 Declaração de variáveis

As declarações de variáveis na Quantix têm a função de reservar espaço em memória e definir o formato dos dados que o programa irá processar. Seguindo o princípio de tipagem explícita, o programador deve indicar se a variável armazenará números inteiros (int), reais (real), textos (txt) ou valores lógicos (lgc).

Funcionamento técnico baseado na linearidade:





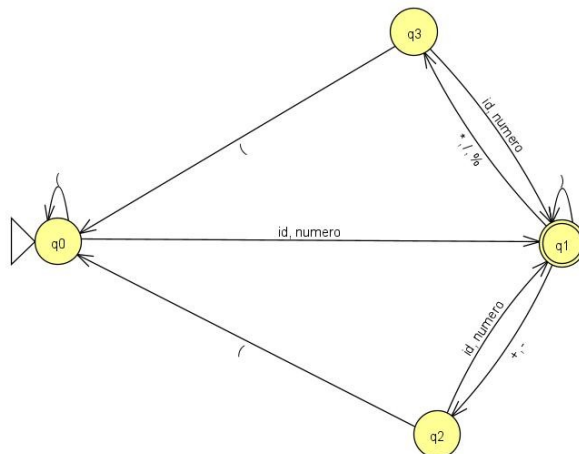
2.1.1.1.1 Cálculos aritméticos

As expressões aritméticas na Quantix são projetadas para suportar precedência através de parênteses e recursão, sendo classificadas como Tipo 2 (Livre de Contexto). Isso permite que o compilador realize o reconhecimento sintático de operações complexas utilizando uma estrutura de pilha.

Estrutura e Regras de Formação

As expressões são formadas por uma sequência alternada de termos e operadores matemáticos. Para ser considerada válida, a expressão deve obrigatoriamente começar com um termo e terminar com um delimitador de fim de instrução (ponto e vírgula) logo após o último termo.

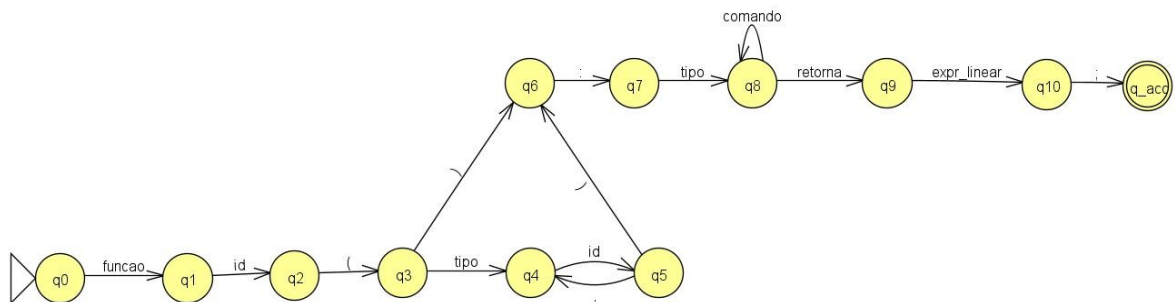
- Termos aceitos: Identificadores de variáveis (id), valores numéricos (numero), cadeias de texto (txt) e literais lógicos (vrdd ou flso).
- Operadores matemáticos: Adição (+), subtração (-), multiplicação (*), divisão (/) e resto da divisão (%).



2.1.1.1.2 Funções

As funções na Quantix são subprogramas projetados para promover a organização lógica e a reutilização de código. Elas permitem que um bloco de instruções seja definido uma única vez e executado sempre que necessário, aceitando parâmetros de entrada e exigindo obrigatoriamente um valor de retorno para o fluxo principal .

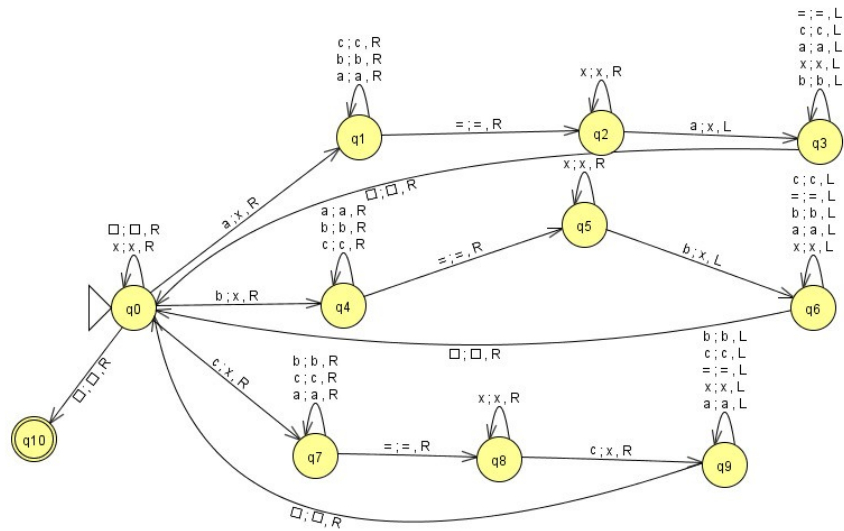
A modelagem das funções em Autômatos Finitos Determinísticos (AFD) segue um fluxo estritamente sequencial e "achatado". O autômato deve validar a assinatura completa da função antes de atingir o estado de aceitação.



3 MÁQUINA DE TURING

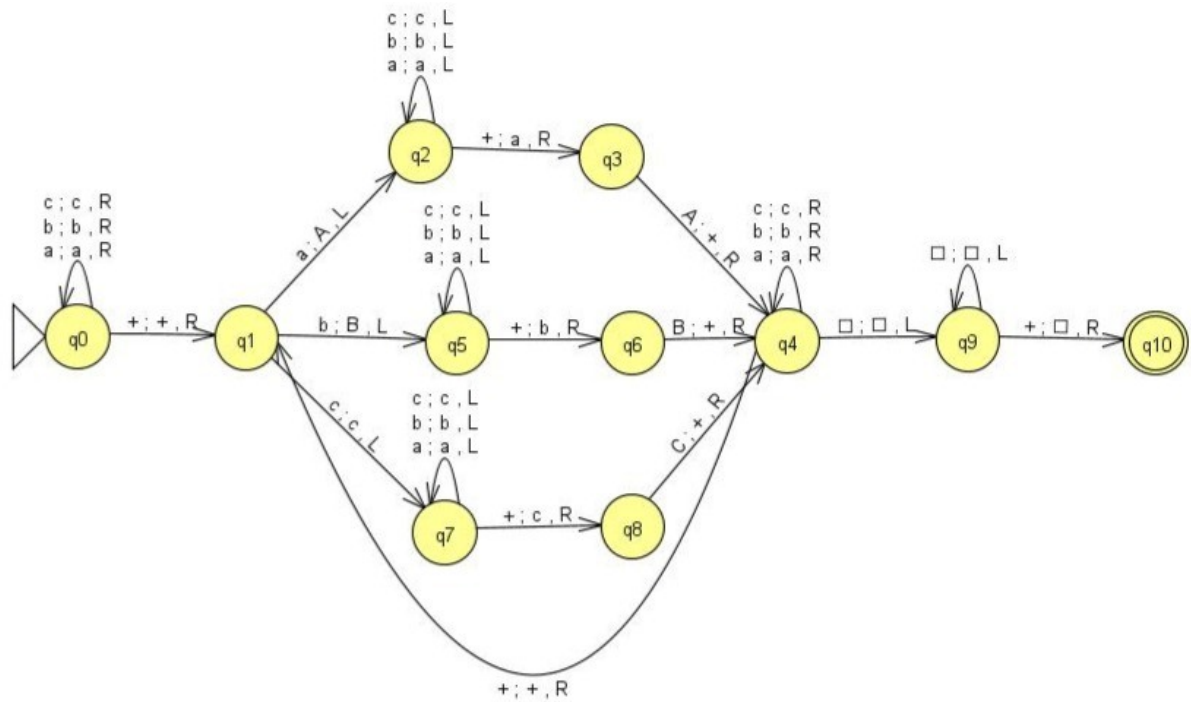
3.1 Comparação de Palavras

Esta funcionalidade implementa uma Máquina de Turing projetada para verificar a identidade entre duas cadeias de caracteres separadas por um delimitador. O objetivo central é validar se a sequência fornecida antes do símbolo de igualdade é exatamente igual à sequência posterior (exemplo: abcc = abcc).



3.1.1 Concatenação de palavras

Esta funcionalidade implementa uma Máquina de Turing projetada para realizar a união de duas cadeias de caracteres, eliminando o operador de soma (+) e fundindo as sequências em uma única sentença contígua (exemplo: transformar palavra1 + palavra2 em palavra1palavra2).



3.1.1.1 Encontrar as letras de uma palavra em um texto

Esta funcionalidade implementa uma Máquina de Turing projetada para realizar uma varredura seletiva em uma cadeia de caracteres, utilizando um padrão definido para mascarar informações. O objetivo é identificar cada caractere individual de uma "palavra-chave" dentro de um "texto-alvo" e substituí-los por um caractere especial (exemplo: transformar and#anderson em &&&#&&erso&).

